BUILT · WORKING

Writing the code coder/

This is the stage that does the actual reimplementation — turning a description of a model into a program that trains it.

WHAT GOES IN, WHAT COMES OUT

INPUT	WHAT HAPPENS	OUTPUT
The facts + the paper	**One call to Claude**	**A runnable script**
stage 2's file, and the original text	write a complete training program	~340 lines, plus a launcher

THE KEY POINTS

- **It writes a whole working program in one go.** Roughly 340 lines — the model, the data loading, the training loop, and the code that reports the final score.
- **It is told exactly which source to trust.** The extracted facts win first; the paper's own text fills gaps; the model's own memory of a famous architecture comes *last* — and whenever it uses that memory, it has to say so in writing.
- **That last rule is the whole trick.** You cannot tell a program "never guess" — something has to be written where a missing number goes. So the rule is *guess visibly*: every guess is listed in the output.
- **Two free checks before the script is accepted.** Is it valid Python at all, and does it accept the settings the next stage will pass it? Both cost nothing and catch broken output early.
- **It also writes a small launcher script** so the next stage can run it without knowing anything about how it works.

HOW WE KNOW IT ISN'T JUST RECITING

THE TEST WE CHOSE, AND WHY

We tested it on **Network In Network** rather than a famous architecture — on purpose. A well-known model would be rebuilt correctly from memory alone, proving nothing. This paper's whole contribution is an unusual layer, its numbers appear nowhere in the text, and its equations include two that belong to *other* methods it argues against.

The script it produced built the unusual layer correctly, skipped both decoy equations by name, and kept the paper's distinctive ending — no standard classifier layer, which every ordinary image model has. It was following the extraction, not its own memory.

THE NETWORK IN NETWORK RUN

343
lines of working code

12
guesses written down

0
standard classifier layers — correct for this paper

What the checks cannot do. They confirm the script is *well-formed*, never that it *works*. Two real bugs have slipped past: code that was perfectly valid but crashed the moment training started, and a script whose own notes described a slightly different model than the one it actually built.

That is exactly why the next stage exists — the only way to find those is to run it.